

Informe de Función Heurística

Proyecto 2 - Knight Energy

Integrantes

Samuel Arenas Valencia	202341928-3743
María Juliana Saavedra	202344035-3743
Libardo Alejandro Quintero	202342181-3743
Juan David Rincón	202332032-3743

1. Contexto del problema

Knight Energy es un juego adversarial entre dos caballos sobre un tablero de ajedrez. La máquina juega con el caballo blanco (*white*) y el jugador humano con el caballo negro (*black*). El objetivo de la inteligencia artificial es elegir movimientos que aumenten sus posibilidades de terminar la partida con más puntos que el rival.

Idea central

El algoritmo minimax necesita una forma de valorar los estados del juego. Cuando el estado es final, se usa una función de utilidad. Cuando el estado no es final pero se alcanzó la profundidad límite del árbol, se usa una función heurística ponderada.

2. Función de utilidad

La función de utilidad evalúa estados terminales, es decir, posiciones en las que la partida ya terminó. Como la inteligencia artificial corresponde al caballo blanco, los valores positivos favorecen a la máquina y los valores negativos favorecen al jugador.

$$U(s) = \begin{cases} 10000 + (P_w - P_b), & \text{si gana white} \\ -10000 + (P_w - P_b), & \text{si gana black} \\ 0, & \text{si hay empate} \end{cases}$$

Donde:

- P_w representa los puntos de la IA.
- P_b representa los puntos del jugador.
- $P_w - P_b$ representa la diferencia de puntos.

El valor 10000 se usa para que ganar o perder tenga más importancia que cualquier ventaja parcial. Así, minimax prefiere una victoria real sobre una posición intermedia aparentemente favorable.

3. Función heurística ponderada

El árbol minimax tiene profundidad limitada según la dificultad seleccionada: profundidad 2 para principiante, profundidad 4 para amateur y profundidad 6 para experto. Por eso, muchas veces el algoritmo no alcanza a llegar hasta un estado terminal. En esos casos, se necesita estimar qué tan bueno es un estado intermedio.

La heurística implementada es:

$$H(s) = 100(P_w - P_b) + 10(E_w - E_b) + 5(M_w - M_b) \\ + 20(A_w^* - A_b^*) + 8(A_w^e - A_b^e) + B(s)$$

Donde:

- $E_w - E_b$ es la diferencia de energía.
- $M_w - M_b$ es la diferencia de movilidad, medida como cantidad de movimientos legales disponibles.
- $A_w^* - A_b^*$ es la diferencia entre el valor de estrellas alcanzables en el siguiente movimiento.
- $A_w^e - A_b^e$ es la diferencia entre el valor de casillas de energía alcanzables en el siguiente movimiento.
- $B(s)$ es la bonificación o penalización por bloqueo.

4. Penalización por bloqueo

La regla del juego indica que si un jugador no tiene energía suficiente para moverse, pierde el turno y se le descuentan 3 puntos. Por esta razón, la heurística también debe anticipar estados donde un jugador queda sin movimientos efectivos.

$$B(s) = \begin{cases} -50, & \text{si white queda bloqueado} \\ +50, & \text{si black queda bloqueado} \\ 0, & \text{si ninguno queda bloqueado} \end{cases}$$

Importancia estratégica

Si la IA queda bloqueada, el estado se considera peligroso y el valor heurístico disminuye. Si el jugador queda bloqueado, el estado se considera favorable y el valor heurístico aumenta.

5. Justificación de los pesos

Componente	Peso	Justificación
Diferencia de puntos	100	Es el factor más importante porque el ganador se decide por puntos.
Diferencia de energía	10	La energía permite seguir moviéndose y evita perder turnos.
Diferencia de movilidad	5	Más movimientos disponibles significan más opciones estratégicas.
Estrellas alcanzables	20	Las estrellas representan oportunidades inmediatas de obtener puntos.
Energía alcanzable	8	La energía ayuda a sostener la partida, pero no da puntos directamente.
Bloqueo	50	Anticipa el riesgo de perder turnos y recibir penalizaciones.

6. Fragmento de implementación

```
1 def heuristica_utility_function(self):
2     if self.is_end_game():
3         return self.utility_function()
4
5     white_moves = self.get_valid_moves("white") if self.
6         can_player_move("white") else []
7     black_moves = self.get_valid_moves("black") if self.
8         can_player_move("black") else []
9
10    points_diff = self.white_points - self.black_points
11    energy_diff = self.white_energy - self.black_energy
12    mobility_diff = len(white_moves) - len(black_moves)
13
14    return (
15        100 * points_diff
16        + 10 * energy_diff
17        + 5 * mobility_diff
18        + 20 * star_options_diff
19        + 8 * energy_options_diff
20        + blocked_score
21    )
```

7. Integración con minimax

La función heurística se usa cuando minimax alcanza la profundidad máxima y el juego todavía no ha terminado. En cambio, si el estado ya es terminal, se usa la función de utilidad.

Criterio de decisión

En los turnos de *white*, minimax maximiza el valor del estado. En los turnos de *black*, minimax minimiza el valor del estado. De esta forma, la máquina asume que el jugador intentará responder con el movimiento más perjudicial para la IA.

8. Conclusión

La heurística ponderada permite que la IA tome decisiones más estratégicas en estados no terminales. La función no solo considera los puntos actuales, sino también la energía, la movilidad, las oportunidades inmediatas de capturar estrellas o recuperar energía, y el riesgo de quedar bloqueado. Esto produce una evaluación más completa del estado del juego y mejora la calidad de las decisiones tomadas por minimax.